

I/O Techniques: Interrupts, Polling, DMA, and I/O Processors

Week 8 — Who Watches the Device While It Works?

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 11, 2026

Where We Left Off (Week 7)

Last week: controlling the chip's own internals

- ▶ Hardwired vs. microprogrammed control — the control unit's two implementations; the latter stores control words in CM and sequences through them with MPC.
- ▶ Both answer the same question: *what does the CPU decree, every cycle?*

The gap we left open

The CPU's commands only reach devices on its own datapath and buses. Nothing yet says how a keypress, or a 20 KB block from a disk, actually gets *into* that CPU's memory [1].

Roadmap: Three Questions, One Thread

1. **The I/O problem** — why devices cannot simply be read like memory: the speed mismatch, and what **synchronous** vs. **asynchronous** I/O means about who waits for whom.
2. **Four techniques** — **polling**, **interrupts**, **DMA**, and the **I/O processor**. Each is an answer to “who watches the device, and what does the CPU pay for that?”
3. **Selecting a technique** — pick one by latency and throughput requirements, not taste.

Running thread

We move data two ways today — *bytes* off a keyboard ($< \sim 10$ bytes/s of human typing) and *blocks* off a disk (megabytes/s) — and the thread is always: **how many CPU cycles does each technique burn per byte moved?**

Today: the CPU is fast and devices are slow — how do we get data from one to the other without wasting the fast one?

Why Can't We Just Read an I/O Device Like Memory?

Three reasons a device needs an I/O module, not a direct wire

- ▶ **Too many kinds of devices:** a keyboard, a disk, a sensor each need their own control logic — too much to build into the CPU itself.
 - ▶ **Speed mismatch, both ways:** some devices are far slower than memory (a human types one byte every ~ 100 ms, i.e. 10^8 **CPU cycles** at 1 GHz); others transfer faster than a single memory access can keep up with.
 - ▶ **Different data formats:** devices use their own word sizes and codes, not the CPU's native format.
-
- ▶ A memory read is *synchronous*: fixed timing, so the CPU always knows exactly when to sample.
 - ▶ A device is *asynchronous*: it becomes ready at an unpredictable instant. Forcing fixed timing on it would waste cycles waiting “just in case.”

[2]

Where Do I/O Registers Live? Memory-Mapped vs. Isolated I/O

Memory-mapped I/O

Device registers share the CPU's memory address space. **LOAD/STORE** reach them exactly like $M[EA]$ from Week 5 — no new opcodes, but those addresses leave memory's own range.

Isolated I/O (I/O-mapped)

Device registers live in a *separate* address space, reached only by dedicated **IN/OUT** instructions. Memory keeps its full range, at the cost of two extra opcodes and a second decode path.

- ▶ Either way, the same *data/status/control* registers exist — only *how the CPU names them* changes.
- ▶ LEGv8/ARM (Weeks 4–5) is memory-mapped throughout; x86 keeps a small isolated space for legacy devices.

[1]

Definition: Synchronous vs. Asynchronous I/O

Synchronous I/O (bus-synchronized)

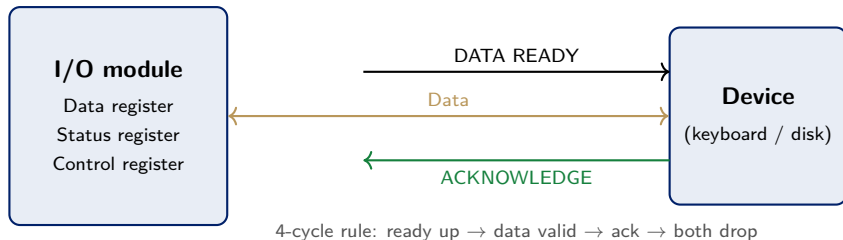
The transfer uses a shared clock; both sides know the timing of data validity in advance (e.g. a memory read on the system bus). No extra agreement is needed per transfer — but the device must actually meet that fixed timing.

Asynchronous I/O (handshake)

No shared assumption about *when*; instead a **handshake** synchronizes each transfer. One control line, **DATA READY**, is raised by the sender when data is valid; another, **ACKNOWLEDGE**, is raised by the receiver once it has taken the data. Slow device or fast, each word is agreed individually.

New notation: **DATA READY/ACKNOWLEDGE** — the two handshake lines, named exactly as in the Intel 8255A's Mode 1 handshake [2] (added to the course registry).

The Handshake, Drawn



Both sides stay synchronized with *no shared clock* — this is how a microseconds-fast disk interface and a millisecond-slow human interface communicate with the same module [2].

Worked Example: Cost of Synchronous-Only Thinking

Scenario

A 1 GHz CPU wants one byte. Under a *hypothetical synchronous keyboard*, it would wait a fixed, worst-case 100 ms per byte just in case the key takes that long — 10^8 cycles burned with the CPU doing nothing but waiting.

- ▶ With a **handshake**, the CPU is *released* after requesting; the device signals readiness later. This frees the CPU to run other work — which is precisely the opening that interrupts (Act 3) exploit.
- ▶ So the real engineering question is not *when* the byte arrives, but *who is responsible for noticing* that it has. That is the four techniques' job.

Who should notice that the keyboard has a byte? The first answer is the simplest: the CPU itself, checking constantly.

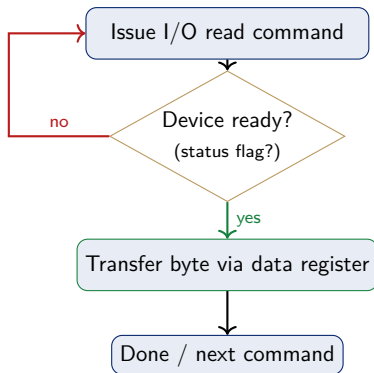
The Simplest Answer: Polling

Programmed I/O (polling)

The CPU issues an I/O command, then **busy-waits**: it repeatedly reads the device's **status flag** until the device marks itself “ready”, then transfers the byte through the data register — all under explicit program control, one instruction at a time.

- ▶ New notation: the **status flag** (often called the F flag, e.g. FGI/FGO in many texts) is a single bit the device sets when it is ready and the program clears after each transfer (added to the registry).
- ▶ No extra hardware beyond the I/O module — the technique is conceptually “just code.” That simplicity is the whole appeal.

The Polling Loop



Everything between “yes” and the next read test is the CPU *looping with nothing useful to do* — the busy-wait that polling is famous for.

Worked Example: Polling a Keyboard, Byte by Byte

Cost arithmetic

1 GHz CPU; human typing at 10 bytes/s $\Rightarrow$ one byte every 100 ms. Polling keeps the CPU in the test loop for essentially all of that 100 ms, because it cannot tell *in advance* that the next key is a long way off.

- ▶ CPU burned while waiting for *one* keystroke: up to 10^8 cycles — all of them useless for other work.
- ▶ Worse, during a long read (*disk seek*), polling holds the whole processor hostage to a device that is orders of magnitude slower than it.
- ▶ Polls are cheap in hardware (just a status-bit test), but they are the most expensive in CPU time — the thread metric for today.

Polling: Trade-offs

Simple hardware, wasteful CPU

- ▶ **Pro:** no interrupt hardware, no DMA controller, no extra context-switch machinery — just a register bit and a loop. Predictable and easy to debug.
- ▶ **Con:** CPU busy-waits even when the device is idle — the cost is paid *whether or not data arrives*.
- ▶ **Con:** with n devices, the CPU must poll all n in turn; the busy-wait cost multiplies, and low-priority devices may starve.

Polling remains standard when the device is *always nearly ready* and there is nothing else to do — e.g. checking a few-frames-ahead media device, or a single dedicated control register.

Polling wastes 10^8 cycles per keystroke — what if the device called the CPU?

The Device Rings the Bell: Interrupt-Driven I/O

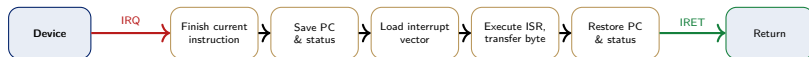
The idea

Instead of the CPU checking the flag forever, the device signals through a dedicated **interrupt request (IRQ)** line whenever it needs attention. The CPU finishes the current instruction, saves state, jumps to an **interrupt service routine (ISR)** that moves exactly the byte that was requested, then returns to what it was doing.

- ▶ The CPU is *released* between transfers — the 10^8 waiting cycles from Act 1 becomes usable work.
- ▶ New notation: **IRQ** (request line), **ISR** (service routine), **IVT** (interrupt vector table, below) — added to the course registry.

[1]

The Interrupt Sequence, Step by Step



Key detail: the CPU checks the IRQ line *at the end of an instruction*, never mid-instruction — so the interrupted program is always resumed at a clean instruction boundary.

How Does the CPU Know Where the ISR Is?

Two mechanisms

- ▶ **Non-vector**: all devices jump to *one* common ISR, which polls each device to find the requester. Simple, but the search adds delay.
- ▶ **Vector**: the device supplies an **interrupt vector** (an index/address of its own ISR) over the bus; the CPU uses it as an entry into the **interrupt vector table (IVT)**, jumping straight to the right routine.

Vectored interrupts are why a fast device can get serviced with just a few extra cycles per interrupt — the device hands the CPU its own branch address.

Worked Example: Interrupt Overhead per Byte

Cost arithmetic (keyboard resume)

One keystroke (100 ms apart) now costs: 1 instruction's finish, save PC/status (≈ 10 – 50 cycles), jump via IVT, ISR + transfer (≈ 30 – 50 cycles), restore and IRET (≈ 10 – 20 cycles). *Total: a few hundred cycles per byte*, versus 10^8 cycles for polling. The CPU keeps the other $\approx 99.99\%$ of the 100 ms.

- ▶ Interrupt-driven I/O turns waiting time into work.
- ▶ But every interrupt still costs a save/restore + jump: if a device interrupts too often (a disk at megabytes/s, byte at a time), the CPU drowns in *overhead* rather than waiting. That motivates DMA (Act 4).

Priorities Among Devices: Who Interrupts First?

Interrupt priority

When several devices interrupt at once, a priority scheme must pick one. Two classic implementations:

- ▶ **Daisy chain**: devices share one IRQ; the acknowledge signal ripples from highest-priority to lowest, and the first device that needs service grabs it — giving *non-uniform* priority by wiring position.
- ▶ **Multiple priority levels**: each level has its own IRQ line; higher levels can preempt lower ones. Safety-oriented sources (e.g. a CPU temperature sensor) get the top level.

Always assign highest priority to interrupts whose delay causes the most damage — damage, not data rate, is the first criterion.

Interrupt-Driven I/O: Trade-offs

Overhead-conscious

- ▶ **Pro:** CPU works between events — the 10^8 -cycle waste of polling disappears for slow devices.
- ▶ **Pro:** handles irregular, asynchronous events naturally — keypresses, packets, sensors.
- ▶ **Con:** per-interrupt save/restore cost. At very high event rates the overhead dominates: billions of small transfers per second through an ISR is not sustainable.
- ▶ **Con:** ISR priority/masking logic adds real hardware complexity.

But a disk delivers megabytes per second — interrupt-per-byte would drown the CPU in overhead. What if the transfer happened without the CPU per byte at all?

The Third Answer: DMA

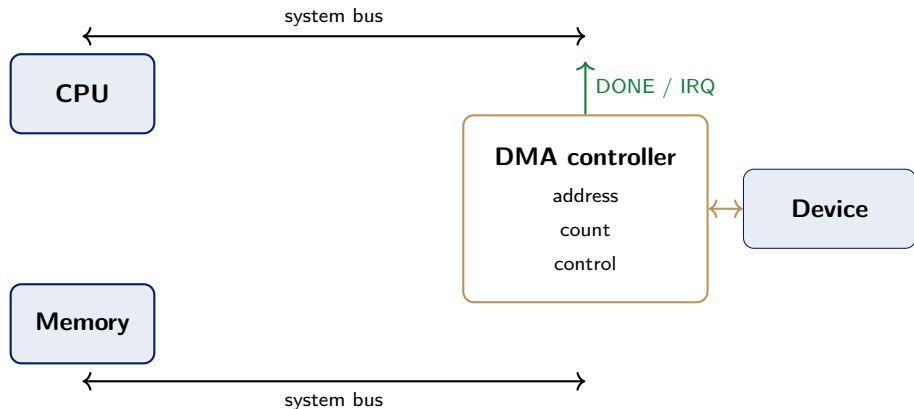
Direct memory access

A **DMA controller (DMAC)** is a dedicated hardware agent that transfers whole blocks *device* $\leftrightarrow$ *memory* while the CPU does other work. The CPU *programs* the DMAC once (device address, memory address, word count, direction), then the DMAC moves the block and raises *one* interrupt when the whole block is done.

- ▶ Compare Act 3: interrupts cost per *byte*; DMA costs per *block*. One interrupt for 20 KB instead of 20,000.
- ▶ New notation: **DMAC**, **burst** (block) mode, **cycle stealing** (below) — added to the registry.

[1]

The DMA Controller at Work



For the transfer, the DMAC becomes **bus master** — it drives the bus itself, reading from the device and writing to memory (or vice versa) in bus-width chunks [1].

DMA Modes: Burst vs. Cycle Stealing

How the DMAC shares the bus

- ▶ **Burst (block) mode**: the DMAC hogs the bus and transfers the whole block in one uninterrupted run — highest throughput, but the CPU is shut out of the bus for the block's duration.
- ▶ **Cycle stealing**: the DMAC transfers one word each time the CPU would otherwise not use the bus (e.g. during an instruction's internal cycles), then releases — the CPU barely notices, at the price of a slower block transfer.
- ▶ **Transparent mode**: transfers only while the CPU is guaranteed idle; lowest interference, lowest bandwidth.

Block-transfer DMA with vectored interrupts maximizes raw I/O bandwidth.

Worked Example: How Much CPU Does DMA Cost?

Cost arithmetic

A 10 MB/s disk transfers 20 KB to memory by DMA. Processor runs at 600 MHz and spends $300 + 900 = 1200$ cycles total to *initiate* and *complete* the transfer. The block takes $20 \text{ KB} / 10 \text{ MB/s} = 2 \text{ ms}$; the processor does $600 \times 10^6 \times 2 \times 10^{-3} = 1.2 \times 10^6$ cycles in that time.

$$\text{CPU time consumed} = \frac{1200}{1.2 \times 10^6} \times 100\% = 0.1\%.$$

- One interrupt at the end — versus 20,000 in interrupt-driven byte mode. DMA owns the block; the CPU pays a flat 0.1%.

DMA: Trade-offs

Throughput vs. system-bus politics

- ▶ **Pro:** bytes move at memory speed without per-byte CPU action — this is how disks, networks, and GPUs actually ship data.
- ▶ **Pro:** single completion interrupt per block.
- ▶ **Con:** the DMAC must arbitrate for the bus; during burst transfers the CPU's own memory access is delayed (the “cycle stealing” tension).
- ▶ **Con:** setup + teardown overhead and real silicon — you do not bolt DMA onto a single keyboard.

But DMA still needs the CPU to set up every transfer. What if an entire I/O program could run on its own processor?

The Fourth Answer: I/O Processor (IOP)

I/O processor (I/O channel)

A small processor, capable of executing **I/O programs**, that manages transfers for *many* devices. The CPU issues one high-level command (start I/O on device #7, block #3); the IOP runs a stored program — issuing device commands, checking status, formatting data — entirely off the main CPU's datapath.

- ▶ **Selector channel**: controls several high-speed devices, but is dedicated to *one* of them at a time until its transfer finishes.
- ▶ **Multiplexor channel**: interleaves data from *several* slower devices at once, instead of finishing one before starting the next.
- ▶ DMA is *fixed-function*; an IOP is *programmed* — its CPU cost drops to a few setup instructions per *job*, not per byte or block.

[2]

DMA vs. IOP, and the Four Techniques Head to Head

	Polling	Interrupt	DMA	IOP
CPU pays	every byte, in a loop	per event	per block	per job
Typical data rate	very low	low-medium	high	very high
Extra hardware	none	IRQ/IVT logic	DMAC	a whole processor
Best for	one simple, fast device	irregular async events	bulk transfers	many/varied devices

One clean rule of thumb: polling for a dedicated always-nearly-ready device; interrupts for irregular events; DMA for bulk blocks; IOP when there is so much I/O that a whole processor is cheaper than babysitting transfers.

Socratic Check: Which Technique, When?

Device droppings to classify

- ▶ A keyboard, key pressed occasionally.
- ▶ A disk shipping 20 KB blocks at 10 MB/s.
- ▶ A memory-mapped sensor that is ready every microsecond.
- ▶ A RAID cabinet running many disks.

Arguments, concept by concept

- ▶ Keyboard: **interrupt** — irregular, long idle gaps.
- ▶ Disk: **DMA** — one interrupt per block, not per byte.
- ▶ Sensor: **polling** — always nearly ready, poll cost tiny.
- ▶ RAID: **IOP** — a per-disk protocol machine is worth it.

The Thread Concludes: Cost per Byte, Four Ways

Technique	CPU cost	Why
Polling (keyboard byte)	$\sim 10^8$ cycles	endless busy-wait
Interrupt (keyboard byte)	$\sim 10^2\text{--}10^3$ cycles	one save/restore + ISR
DMA (20 KB disk block)	1200 cycles / block	flat setup + 1 interrupt
IOP (whole disk job)	a few setup instructions	runs its own program

The one-sentence takeaway

Every technique moves the same bytes; they differ in *how many CPU cycles are spent per byte moved* — polling spends the most, the IOP the least — and you should select the cheapest one that still gets the bytes there in time (the Week 8 learning objective).

Bridge to Week 9

From moving data to keeping it

This week moved bytes from the outside world *into memory*. Week 9 asks how memory itself is organized — caches, the memory hierarchy, and which bytes are worth keeping close. The DMA block you just moved lands in that hierarchy.

- ▶ The throughput you defend with a technique is bounded by the memory system it feeds — DMA is only as fast as main memory allows.
- ▶ Next week's cache question is: *given* a memory you cannot make uniformly fast, where does the data go?

Summary

- ▶ **Synchronous** I/O assumes fixed timing; **asynchronous** I/O handshakes per transfer (*DATA READY/ACKNOWLEDGE*) — the only sane way to talk to devices of wildly different speeds.
- ▶ **Polling** is simplest and cheapest in hardware but wastes the most CPU (10^8 cycles/keypress in our example).
- ▶ **Interrupt-driven I/O** releases the CPU between events and drops the per-byte cost to a few hundred cycles — at the price of ISR / priority hardware.
- ▶ **DMA** moves whole blocks with one completion interrupt — 0.1% CPU in the worked example above; choose **burst** for throughput and **cycle stealing** for CPU friendliness.
- ▶ **IOP** is a programmable processor for whole I/O jobs.
- ▶ **Select on cost per byte vs. latency/throughput** — never on habit.

References



V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.

Computer Organization.

McGraw-Hill, 5th edition, 2002.



William Stallings.

Computer Organization and Architecture: Designing for Performance.

Pearson, 10th edition, 2015.